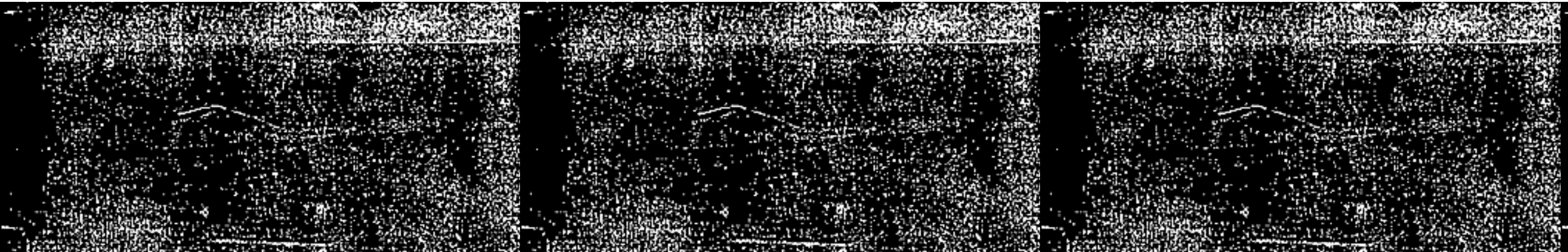
PROGRAMMATION WEB DYNAMIQUE

Benoit Gaudou - benoit.gaudou@ut-capitole.fr

Source : Laurent Perrussel

TEMPLATING



MOTEUR DE TEMPLATE

- Spring (boot) se base sur des moteurs de template pour générer les pages HTML. Le moteurs supportés incluent :
  - JSP
  - FreeMarker
  - Thymeleaf

- Un fichier de template **contiendra des instructions pour générer une page HTML**, **à partir de données** transmises depuis la méthode gérant la route associée.
- En Thymeleaf, the fichier de template est un fichier **HTML valide** : les **instructions sont des attributs de balises HTML**.

EXEMPLE DE TEMPLATE

greetings.html

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
    <p th:text="|Hello, ${name}!|" />
</body>
</html>
```

EXEMPLE DE TEMPLATE

greetings.html

```html
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
    <p th:text="|Hello, ${nameT}!| » />
</body>
</html>
```

```java
@Controller
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("nameT", name);
        return "greetings";
    }
}
```

EXEMPLE DE TEMPLATE

- **La transmission de données entre le contrôleur et le template se fait au travers de la variable `Model model`.**
- Des objets de tout type peuvent être ajoutés dans le model (int, String, List …)

```
@Controller
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        return "greetings";
    }
}
```

greetings.html

```
<body>
    <p th:text="|Hello, ${name}!|" />
</body>
```

EXEMPLE DE TEMPLATE

- Note : l'utilisation de | | pour agréger une variable avec une chaine de caractères.
- **${…}** permet d'évaluer une expression,
  - Une variable : **${name}**
  - L'attribut d'une variable (de type objet): **${student.gender}**
  - Une expression : **${number * 2}**

```
@Controller
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", required=false, defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        return "greetings";
    }
}
```

greetings.html

```
<body>
    <p th:text="|Hello, ${name}!|" />
</body>
```

EXEMPLE D'INSTRUCTIONS

- th:text
- th:each
- th:if
- th:unless
- th:switch
- th:case

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
     <p th:text="|Hello, ${name}!|" />
     <p th:text="${number}" />
     <p th:text="${number*2}" />

     <ol>
          <li th:each="l: ${li}">
          <span th:text="${l}" />
          </li>
     </ol>
</body>
</html>
```

```
<tr th:each="student: ${students}">
    <td th:text="${student.id}" />
    <td th:text="${student.name}" />
</tr>
```

```
<td>
    <span th:if="${student.gender} == 'M'" th:text="Male" />
    <span th:unless="${student.gender} == 'M'" th:text="Female" />
</td>
```

```
<td th:switch="${student.gender}">
    <span th:case="'M'" th:text="Male" />
    <span th:case="'F'" th:text="Female" />
</td>
```

EXEMPLE D'INSTRUCTIONS

- th:each
  - Permet de boucler sur les éléments d'une liste (ou autre)

  - **La boucle répète l'élément sur lequel il est ajouté :**
    - *1er cas :* 1 liste <ol> et plusieurs <li>
    - *2ème cas :* plusieurs <ol> et 1 <li> par <ol>

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
     <ol>
          <li th:each="l: ${li}">
          <span th:text="${l}" />
          </li>
     </ol>

     <ol th:each="l: ${li}">
          <li>
          <span th:text="${l}" />
          </li>
     </ol>
</body>
</html>
```

EXEMPLE D'INSTRUCTIONS

- th:each
  - Permet de boucler sur les éléments d'une liste (ou autre)
  - **La boucle répète l'élément sur lequel il est ajouté :**
    - *1er cas :* 1 liste <ol> et plusieurs <li>
    - *2ème cas :* plusieurs <ol> et 1 <li> par <ol>

```
<!DOCTYPE HTML>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Getting Started: Serving Web Content</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>
<body>
     <ol>
          <li th:each="l: ${li}">
          <span th:text="${l}" />
          </li>
     </ol>

     <ol th:each="l: ${li}">
          <li>
          <span th:text="${l}" />
          </li>
     </ol>
</body>
</html>
```

```
1  <!DOCTYPE HTML>
2  <html>
3  <head>
4      <title>Getting Started: Serving Web Content</title>
5      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"
6  </head>
7  <body>
8      <ol>
9          <li>
10             <span >Paris</span>
11         </li>
12         <li>
13             <span >Marseille</span>
14         </li>
15         <li>
16             <span >Lyon</span>
17         </li>
18         <li>
19             <span >Toulouse</span>
20         </li>
21     </ol>
22
23     <ol>
24         <li>
25             <span >Paris</span>
26         </li>
27     </ol>
28
29     <ol>
30         <li>
31             <span >Marseille</span>
32         </li>
33     </ol>
34
35     <ol>
36         <li>
37             <span >Lyon</span>
38         </li>
39     </ol>
40
41     <ol>
42         <li>
43             <span >Toulouse</span>
44         </li>
45     </ol>
46 </body>
47 </html>
```

EXEMPLE D'INSTRUCTIONS

- th:if
- th:unless

- Ils permettent d'afficher **l'élément auquel elle est appliquée** si une variable est vraie (resp. Fausse).

```
<span th:if="${footer}" th:text="FOOOOOTER" />
<span th:unless="${footer}" th:text="|No Footer|" />
```

```
<div th:if="${footer}">
    <footer>Pied de page footer </footer>
</div>

<div th:unless="${footer}">
    <div class="footer">Pied de page div class footer</div>
</div>
```

EXEMPLE D'INSTRUCTIONS

- th:switch
- th:case

```
<td>
    <span th:if="${student.gender} == 'M'" th:text="Male" />
    <span th:unless="${student.gender} == 'M'" th:text="Female" />
</td>
```

```
<td th:switch="${student.gender}">
    <span th:case="'M'" th:text="Male" />
    <span th:case="'F'" th:text="Female" />
</td>
```

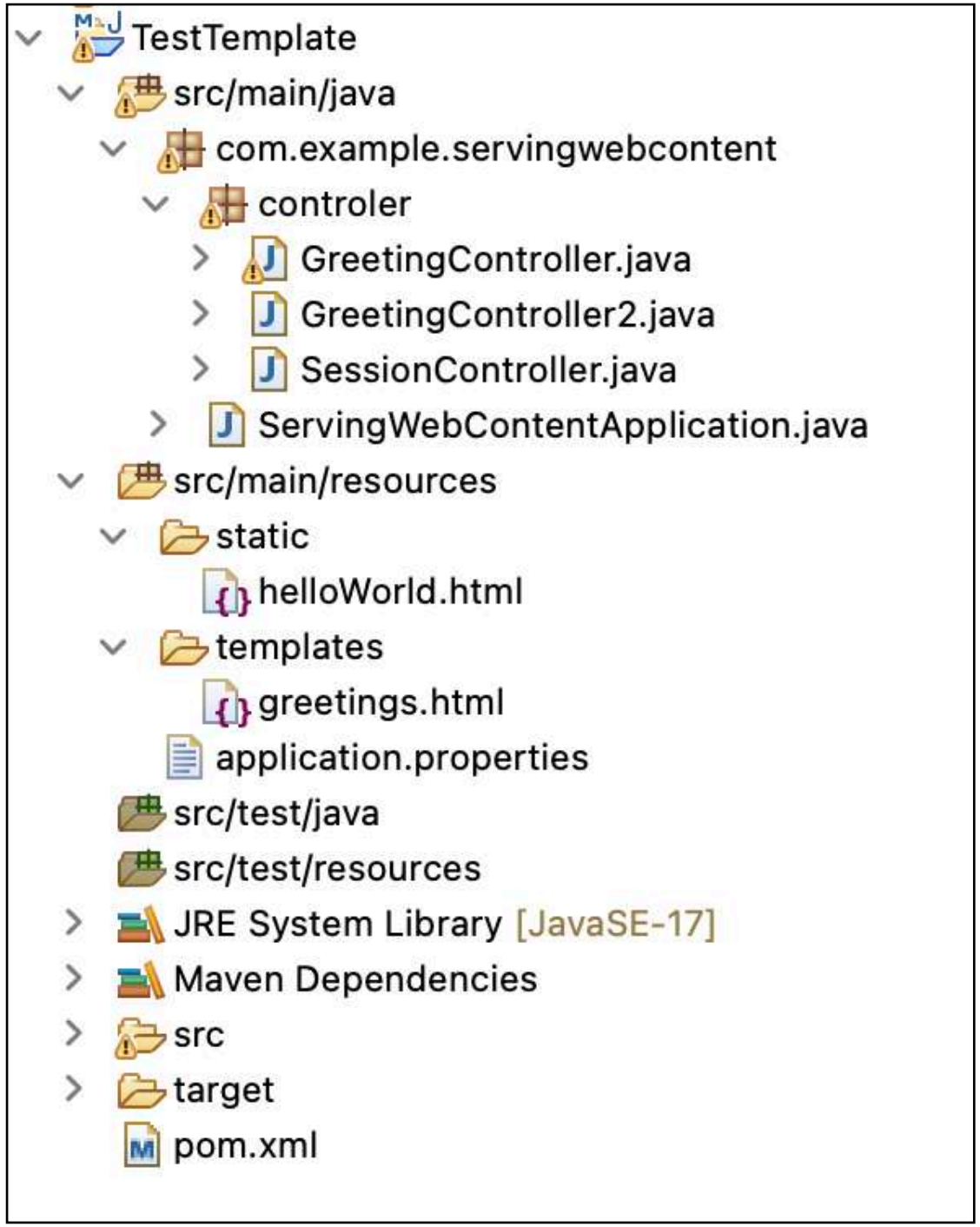
ARCHITECTURE

- Par défaut (avec SpringBoot), les fichiers templates sont placés dans le dossier :
  `src/main/resources/templates`

- Les templates sont des fichiers HTML.

- D'autres pages HTML (sans instructions) sont placées dans le dossier :
  `src/main/resources/static`

- `Src/main/resources` contient également le fichier de configuration
  `application.properties`



APPELLE FICHIERS, ROUTES

- Une méthode affiche un **template** avec:

  `return "nomTemplateFile";`

- Une méthode affiche une **page static** (donc sans évaluer les instructions) avec:

  `return redirect:/fichierHTML.html`

- **`redirect` peut être utiliser avec une route,** et donc appelle une autre méthode du contrôleur.

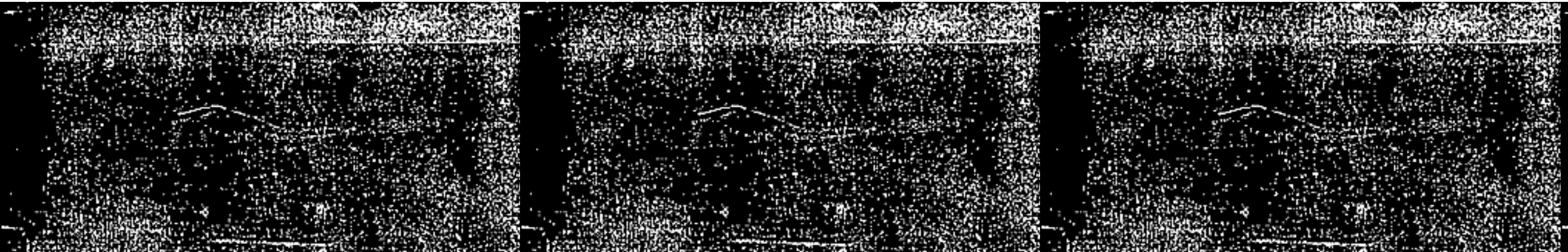
```java
@Controller
public class GreetingController2 {


    @GetMapping("/hw")
    public String hw() {
        return "redirect:/helloWorld.html";
    }


    @GetMapping("/greeting")
    public String greeting(
            @RequestParam(name="name", defaultValue="World") String name,
            Model model) {
        model.addAttribute("name", name);
        model.addAttribute("number", 10);

        ArrayList<Integer> li = new ArrayList<Integer>();
        li.add(23);
        li.add(43);
        li.add(35);
        model.addAttribute("li",li);

        model.addAttribute("footer",false);

        return "greetings";
    }
}
```

REMARQUE

- Les méthodes associées à une route peuvent combiner @PathVariable, @RequestParam et model…

```
    @RequestMapping("/login")
    public String login(
          @RequestParam(name="name", required=true) String name,
          @RequestParam(name="givenname", required=true) String givenName,
           Model model) {
        session.setAttribute("sName", name);
        session.setAttribute("sGivenName", givenName);
        return "index";
    }
```

SESSIONS



INTRODUCTION

- Après l'envoi de la réponse HTTP, client et serveur ne sont plus connectés.
  - Il n'est donc pas possible d'envoyer des données entre pages du site. ex: panier, information de connection …

- L'objectif des **sessions** est d'enregistrer en mémoire des données qui vont être accessibles sur plusieurs pages, pendant une durée donnée.
- Ces informations proviennent de choix / sélection de l'utilisateur lors de sa visite du site.

SESSIONS

- Plusieurs alternatives techniques existent pour stocker ces données :
  - En mémoire sur le serveur
  - Cookie côté client
  - Base de données
  - …

- Ces sessions peuvent être limitées dans le temps.
  - Le *timeout* des session peut être fixé dans le fichier de configurations `application.properties`

```
server.servlet.session.timeout=30m
```

SESSION HTTP PAR DÉFAUT : EN MÉMOIRE

- SpringBoot utilise Spring Session pour gérer des sessions HTTP.
- Par défaut, SpringBoot stocke les données de session en mémoire en utilisant les `HttpSession` de Tomcat.

- Les sessions sont accessibles directement en passant un attribut supplémentaire aux méthodes associées aux routes.

```
@RequestMapping("/setSession")
public String setSession(HttpSession session) {
    session.setAttribute("user", "JohnDoe");
    return "Session attribute set";
}

@RequestMapping("/getSession")
public String getSession(HttpSession session) {
    return "User: " + session.getAttribute("user");
}
```

UTILISATION DES SESSIONS

- Une variable de session est une sorte de **dictionnaire** contenant des données identifiées par une clé (de type string).
- `session.setAttribute(String, Object);`

  Attache un objet à cette session, en utilisant le nom spécifié. Si un objet du même nom est déjà lié à la session, l'objet est remplacé.
- `session.removeAttribute(String);`

  Supprime de cette session l'objet lié au nom spécifié. Si la session n'a pas d'objet lié au nom spécifié, cette méthode ne fait rien.
- `session.getAttribute(String)`

  Renvoie l'objet lié au nom spécifié dans cette session, ou **null** si aucun objet n'est lié à ce nom.

EXEMPLE D'UTILISATION DES SESSIONS

```
@RequestMapping("/getSession")
public String getSession(HttpSession session) {

     String elts = "";
    Enumeration<String> enume = session.getAttributeNames();
    while (enume.hasMoreElements()) {
         String key = enume.nextElement();
         elts += key + session.getAttribute(key) + " - ";
    }

 return elts;
}
```

REMARQUE

- Les méthodes associées à une route peuvent combiner HttpSession, @PathVariable, @RequestParam et template…

```
@RequestMapping("/login")
public String login(
        @RequestParam(name="name", required=true) String name,
        @RequestParam(name="givenname", required=true) String givenName,
        HttpSession session,
        Model model) {
    session.setAttribute("sName", name);
    session.setAttribute("sGivenName", givenName);
    return "index";
}
```

TEMPLATE ET SESSION

- La variable de session est directement accessible depuis le template.

```
<ol>
    <li th:each="l: ${session}">
     <span th:text="${l}" />
    </li>
</ol>

<span th:text="${session.user}">
```

1. user1=toto
2. user2=titi
3. user=JohnDoe
4. user3=teuteu

JohnDoe